Assignment 1:
Exploring OpenGL Programming

NAME: SHU HAOYU
STUDENT NUMBER: 2024533012
EMAIL: SHUHY2024@SHANGHAITECH.EDU.CN

1 INTRODUCTION

I finished below all tasks using the OpenGL and Arietta math library. OpenGL is a machine which can remember current status, modify state according to the input content and state, and can also get the output.

- Task1: load mesh objects from files and draw the meshes.
- Task1: render objects with a Phong lighting model.
- Task2: manipulate the camera using both the keyboard and mouse. Support keyboard controls for translation and rotation, and mouse controls for looking around so that you can navigate the virtual scene.
- Bonus1:support moving objects in the scene.
- Bonus2: support multiple objects in the scene.
- Bonus3: support multiple lights in the fragment shader, with different types of lights (such as point lights and spotlights) and different colors.
- Bonus4: add a skybox to the scene to make the scene more realistic.

2 IMPLEMENTATION DETAILS

2.1 load mesh objects from files and draw

I implemented mesh loading and drawing in `Mesh.cpp`. The constructor `Mesh(path)` takes the mesh file path as its parameter. I use `PathOf(target, depth)` to find the file. Here, `target` is the requested path, and `depth` is the maximum number of parent-directory levels to search, with a default value of 5. The function returns the absolute file path as a string, or throws an exception if the file cannot be found. Then I open it with `std::ifstream` and use the input operator >> to read the data in order.

First, I read the numbers of positions, normals, and triangles from the beginning of the `.object` file. I use two temporary vectors to store the positions and normals. Then I read each triangle record, which contains three pairs of position and normal indices. For each pair, I combine the corresponding position and normal into a `Vertex` and add it to `vertices_`. I also store the indices of these new vertices in `indices_`. Therefore, each final vertex contains both its position and its normal, although I still need a temporary vector when reading the normals.

I obtain the VAO ID with `glGenVertexArrays` after loading the data. I create the VBO and EBO IDs with `glGenBuffers`. Their first parameter is the number of objects to create, and the second is the address where the IDs are stored. I use `glBindVertexArray` with the VAO ID to select its configuration. I also use `glBindBuffer`, whose parameters are the buffer target and buffer ID. The VBO uses `GL_ARRAY_BUFFER`, and the EBO uses `GL_ELEMENT_ARRAY_BUFFER`.

I upload the data with `glBufferData`. Its four parameters are the buffer target, the data size in bytes, the address of the data, and a usage hint. For the vertices, I use `vertices_.size()` multiplied by `sizeof(Vertex)` as the byte size. For the triangle indices, I use the number of stored index triples multiplied by the size of one triple. I choose `GL_STATIC_DRAW` because the uploaded mesh data stays unchanged while the program runs.

Then I describe the vertex layout with `glVertexAttribPointer`. Its parameters are the attribute location, number of components, component type, integer normalization flag, stride in bytes, and offset in the VBO. I use location 0 for the position and location 1 for the normal. Both have three `GL_FLOAT` components, with `GL_FALSE` for integer normalization and `sizeof(Vertex)` as the stride. The position offset is zero. For the normal, I use `offsetof` with `Vertex` and its `normal` member to get the offset. Finally, `glEnableVertexAttribArray` takes an attribute location and enables it. The VAO records these settings and the associated buffers.

In `Main()`, I create a `Mesh` object by passing the file path to its constructor. In the rendering loop, I call its `Draw()` function, which has no parameters. It binds the VAO and calls `glDrawElements`. The four arguments specify the primitive type, number of scalar indices, index type, and byte offset in the EBO. I use `GL_TRIANGLES`, three indices per stored triangle, `GL_UNSIGNED_INT`, and offset 0. The mesh file is loaded once, rather than read again in every frame.

Before drawing a frame, I call `glClearColor` with red, green, blue, and alpha values to set the clear color. Then `glClear` takes a mask containing `GL_COLOR_BUFFER_BIT` and `GL_DEPTH_BUFFER_BIT` to clear both buffers. I present the frame with `glfwSwapBuffers`. Its single parameter identifies the window whose back buffer is presented.

2.2 render objects with a Phong lighting model

I use `model.vert` and `model.frag` for Phong lighting. The constructor `Shader(vsPath, fsPath)` takes the paths of the vertex and fragment shader files. In `Shader.cpp`, I use `glCreateShader` with a shader type to create each shader. Then `glShaderSource` receives the shader ID, number of source strings, address of the source-string pointer, and source lengths. I pass one string and use `nullptr` for the lengths because the string is null-terminated. I call `glCompileShader` with the shader ID to compile it.

Next, I call `glCreateProgram`, which takes no parameters and returns a program ID. I use `glAttachShader` with the program ID and shader ID, and then `glLinkProgram` with the program ID. I check

© 2026
Computer Graphics I Course Assignment

compilation with `glGetShaderiv` and `GL_COMPILE_STATUS`, and linking with `glGetProgramiv` and `GL_LINK_STATUS`. These queries take an object ID, the property to query, and a pointer for the result. If a step fails, I use `glGetShaderInfoLog` or `glGetProgramInfoLog` to read its diagnostic log. Their parameters are the object ID, output buffer size, an optional pointer for the written length, and the output buffer. Then I throw an exception.

Before setting uniforms, I call `Shader::Use()`, which activates the program through `glUseProgram`. I pass uniforms through `Shader::Set`. Its parameters are `name` and `value`: the uniform name and the value to upload. I use it for the matrices, camera position, material color, and lighting parameters. The member function `GetUniformLocation` takes a uniform name and finds its location using the program ID and that name. The `Set` overload chooses the corresponding uniform function. For example, `glUniformMatrix4fv` takes the location, matrix count, transpose flag, and data pointer. I pass one matrix with `GL_FALSE` to upload it without transposing it.

First, the vertex shader uses the model matrix to transform the vertex position into world space. It then applies the view and projection matrices to obtain the clip-space position. I also transform the normal using the upper-left three-by-three part of the model matrix. This works for the translations and positive uniform scaling used in my scene, after normalization in the fragment shader. Non-uniform scaling would require an inverse-transpose normal matrix.

The MVP transformation. I use column vectors and a right-handed coordinate system. A local vertex is written as the homogeneous point $\mathbf{p}_o = (x, y, z, 1)^\mathsf{T}$. The last component is 1 for a point and 0 for a direction, so translation changes points but not directions. The three matrices act in this order:

$$\begin{aligned} \mathbf{p}_w &= M\mathbf{p}_o, \\ \mathbf{p}_v &= V\mathbf{p}_w, \\ \mathbf{p}_c &= P\mathbf{p}_v = PVM\mathbf{p}_o. \end{aligned} \tag{1}$$

Here, the subscripts mean object, world, view, and clip space. Although I call this the MVP transformation, the product in my shader is PVM, because the rightmost matrix acts first. After clipping, perspective division gives normalized device coordinates (NDC):

$$\mathbf{p}_{\text{ndc}} = \left(\frac{x_c}{w_c}, \frac{y_c}{w_c}, \frac{z_c}{w_c} \right). \tag{2}$$

The viewport then maps these coordinates to the window. Therefore, the output of PVM is a clip-space point, and it still needs division by w_c before the viewport mapping.

Then the fragment shader receives the interpolated world position and normal. I normalize the normal, the direction from the surface to the light, and the direction from the surface to the camera. All these quantities are in world space, so I can use them together in the lighting calculation.

The Phong model has three parts: ambient, diffuse, and specular lighting. The ambient part gives the object a basic color even when it does not face a light, with `ka` controlling its strength. For the diffuse part, I use `dot(N, L)` to calculate the dot product of the unit normal and the direction toward the light. I use `max` with zero to remove negative values. For the specular part, `reflect(-L, N)` returns the reflected direction, where `-L` is the incoming direction and `N` is the unit normal. I compare this result with the viewing direction and use `pow` to raise the nonnegative dot product to `s`. Its first argument is the base, and its second is the exponent. The coefficient `ks` controls the highlight strength, and `s` controls how concentrated it is. I also set the specular contribution to zero when the surface faces away from the light. Finally, I add the lighting contributions to get the fragment color.

For a world-space surface point $\mathbf{x}$, light position $\mathbf{l}_j$, and camera position $\mathbf{e}$, I use unit vectors

$$\boldsymbol{\ell}_j = \frac{\mathbf{l}_j - \mathbf{x}}{\|\mathbf{l}_j - \mathbf{x}\|}, \qquad \mathbf{v} = \frac{\mathbf{e} - \mathbf{x}}{\|\mathbf{e} - \mathbf{x}\|}. \tag{3}$$

I assume these differences are nonzero. Since the normal $\mathbf{n}$ is also a unit vector, $\mathbf{n} \cdot \boldsymbol{\ell}_j$ is the cosine of the angle toward the light. This gives the diffuse factor. Reflecting the incoming vector $-\boldsymbol{\ell}_j$ across the surface gives

$$\begin{aligned} d_j &= \max(\mathbf{n} \cdot \boldsymbol{\ell}_j, 0), \\ \boldsymbol{\rho}_j &= 2(\mathbf{n} \cdot \boldsymbol{\ell}_j)\mathbf{n} - \boldsymbol{\ell}_j, \\ h_j &= \begin{cases} [\max(\boldsymbol{\rho}_j \cdot \mathbf{v}, 0)]^s, & d_j > 0, \\ 0, & d_j = 0. \end{cases} \end{aligned} \tag{4}$$

Thus, the color returned by `light` is

$$F_j = d_j(\mathbf{C}_j \odot \mathbf{C}_m) + k_s h_j \mathbf{C}_j. \tag{5}$$

Here, $\mathbf{C}_j$ is the light color, $\mathbf{C}_m$ is the material color, and $\odot$ means component-wise RGB multiplication. The exponent s is the Phong shininess value, rather than the scale factor used by `scale`. This is the Phong lighting model used in my shader.

2.3 manipulate the camera using both the keyboard and mouse

I implemented camera control in `Main.cpp` and use the functions in `Math.cpp` to build the required matrices. I store the camera position in `eye` and its viewing direction in `direction`. I process input events with `glfwPollEvents`, which takes no parameters. To check a key, I use `glfwGetKey(window, key)`. Its parameters identify the window and key, and I compare the result with `GLFW_PRESS`. I also use `glfwGetTime()`, which returns elapsed time in seconds, to calculate the time between frames.

For translation, W and S move the camera along its viewing direction. A and D move it sideways. I use `direction.Cross(up)` to obtain the cross product of the viewing direction and world up vector, and divide it by its `Norm()` to obtain a unit right vector. Space and Left Ctrl move the camera up and down along the world vertical axis. I multiply the movement speed by the time between frames, so the displacement depends on elapsed time rather than the number of frames.

For rotation, Q and E change yaw, while Z and C change pitch. I register `MouseCallback` through `glfwSetCursorPosCallback`, passing the window and callback function. The callback receives `window`, `xpos`, and `ypos`; the last two are cursor coordinates in the window's content area. I compare them with the previous coordinates to obtain the movement. I multiply this movement by a sensitivity value and add it to yaw and pitch. The first callback only records the mouse position, which avoids a sudden change when mouse control starts. I limit pitch to $[-89°, 89°]$ to avoid looking

exactly along the world up axis. I use `glfwSetInputMode` with the window, `GLFW_CURSOR`, and `GLFW_CURSOR_DISABLED` to hide and capture the cursor.

In each frame, I rebuild the viewing direction from the initial forward vector and the stored angles. I use `rotate(theta, axis)`, which returns a four-by-four rotation matrix. Here, `theta` is an angle in degrees, and `axis` is the rotation axis, which the function normalizes internally. I use the three-by-three rotation part for the direction vector. Yaw rotates it around the world up axis, and pitch rotates it around the current right axis.

Deriving `rotate`. First, I convert the input angle to radians, $\theta_r = \theta\pi/180$, and normalize the nonzero axis to $\mathbf{a}$. To rotate a vector $\mathbf{b}$, I split it into a part along the axis and a part perpendicular to it:

$$\mathbf{b}_{\parallel} = \mathbf{a}(\mathbf{a}\cdot\mathbf{b}), \qquad \mathbf{b}_{\perp} = \mathbf{b} - \mathbf{b}_{\parallel}. \tag{6}$$

The parallel part stays unchanged. The perpendicular part rotates in its plane, where $\mathbf{a}\times\mathbf{b}$ gives the second perpendicular direction. Therefore,

$$\mathbf{b}' = \mathbf{b}_{\parallel} + \cos\theta_r\,\mathbf{b}_{\perp} + \sin\theta_r\,(\mathbf{a}\times\mathbf{b}). \tag{7}$$

After substituting the two parts and collecting terms, I obtain Rodrigues' rotation matrix:

$$R = \cos\theta_r I + (1-\cos\theta_r)\mathbf{a}\mathbf{a}^{\mathsf{T}} + \sin\theta_r[\mathbf{a}]_{\times}, \tag{8}$$

$$[\mathbf{a}]_{\times} = \begin{pmatrix} 0 & -a_z & a_y \\ a_z & 0 & -a_x \\ -a_y & a_x & 0 \end{pmatrix}. \tag{9}$$

This matrix satisfies $[\mathbf{a}]_{\times}\mathbf{b} = \mathbf{a}\times\mathbf{b}$. In my code, `First`, `Second`, and `Third` are the three terms of R. I put R in the upper-left part of a four-by-four identity matrix. The angle follows the right-hand rule; when $\theta = 0$, the result is the identity matrix.

For yaw ψ and pitch ϕ, the direction update can be written as

$$\begin{aligned} \mathbf{d}_y &= R(\psi, \mathbf{u}_0)\mathbf{d}_0, \\ \mathbf{r}_y &= \frac{\mathbf{d}_y\times\mathbf{u}_0}{\|\mathbf{d}_y\times\mathbf{u}_0\|}, \\ \mathbf{d} &= R(\phi, \mathbf{r}_y)\mathbf{d}_y. \end{aligned} \tag{10}$$

Here, $\mathbf{d}_0 = (0,0,-1)$ is the initial forward direction and $\mathbf{u}_0 = (0,1,0)$ is world up. This matches my order of yaw first and pitch second. Rotations preserve vector length, so the direction remains unit length in exact arithmetic.

Finally, I call `lookAt(eye, center, up)` to update the view matrix. The parameters are the camera position, a world-space point to look at, and a reference up vector. In my call, `center` is the point `eye + direction`, rather than the direction alone. I also use `perspective` with `fovY`, `aspect`, `nearplane`, and `farplane` to obtain the projection matrix. These parameters are the vertical field of view in degrees, window width divided by height, and positive distances to the near and far clipping planes. My settings are 50°, 1280/720, 0.1, and 100. Changing the view matrix changes the camera view without modifying the mesh data.

Deriving `lookAt`. I use $\mathbf{e}$ for `eye`, $\mathbf{c}$ for the target `center`, and $\mathbf{u}_0$ for the reference up vector. First, I build the camera's forward, right, and corrected up directions:

$$\begin{aligned} \mathbf{f} &= \frac{\mathbf{c}-\mathbf{e}}{\|\mathbf{c}-\mathbf{e}\|}, \\ \mathbf{r} &= \frac{\mathbf{f}\times\mathbf{u}_0}{\|\mathbf{f}\times\mathbf{u}_0\|}, \\ \mathbf{u} &= \frac{\mathbf{r}\times\mathbf{f}}{\|\mathbf{r}\times\mathbf{f}\|}. \end{aligned} \tag{11}$$

The target must differ from the eye, and the reference up vector must not be parallel to the forward direction. My function checks these cases before dividing. The camera's positive Z axis is $-\mathbf{f}$, so points in front of it have negative view-space Z.

Then I subtract the camera position from a world point $\mathbf{x}$. Its camera coordinates are the projections of $\mathbf{x}-\mathbf{e}$ onto the three camera axes:

$$\begin{aligned} x_v &= \mathbf{r}\cdot(\mathbf{x}-\mathbf{e}), \\ y_v &= \mathbf{u}\cdot(\mathbf{x}-\mathbf{e}), \\ z_v &= -\mathbf{f}\cdot(\mathbf{x}-\mathbf{e}). \end{aligned} \tag{12}$$

Expanding these dot products gives the view matrix:

$$V = \begin{pmatrix} r_x & r_y & r_z & -\mathbf{r}\cdot\mathbf{e} \\ u_x & u_y & u_z & -\mathbf{u}\cdot\mathbf{e} \\ -f_x & -f_y & -f_z & \mathbf{f}\cdot\mathbf{e} \\ 0 & 0 & 0 & 1 \end{pmatrix}. \tag{13}$$

The vectors `v1` through `v4` in my function store its four columns. As a check on the formula, the eye maps to $(0,0,0,1)$, and the point $\mathbf{e}+\mathbf{f}$ maps to $(0,0,-1,1)$.

Deriving `perspective`. I write n and f for the positive near and far distances, with $0 < n < f$, and a for the aspect ratio. Let $\alpha = \texttt{fovY}\,\pi/180$, with $0 < \alpha < \pi$. From the right triangle between the eye and the top of the near plane, its half-height is $h = n\tan(\alpha/2)$ and its half-width is ah.

For a view-space point (x_v, y_v, z_v) in front of the camera, its positive depth is $d = -z_v$. Similar triangles give $x_{\text{near}} = nx_v/d$ and $y_{\text{near}} = ny_v/d$. Dividing by the near-plane half-width and half-height gives

$$x_{\text{ndc}} = \frac{qx_v}{ad}, \qquad y_{\text{ndc}} = \frac{qy_v}{d}, \tag{14}$$

where $q = 1/\tan(\alpha/2)$. I can obtain these expressions by setting $x_c = qx_v/a$, $y_c = qy_v$, and $w_c = -z_v = d$, then dividing by w_c.

Next, I choose $z_c = Az_v + B$. OpenGL's NDC depth should be -1 on the near plane and $+1$ on the far plane. Substituting $z_v = -n$ and $z_v = -f$ gives

$$\begin{aligned} \frac{-An+B}{n} &= -1, \\ \frac{-Af+B}{f} &= 1. \end{aligned} \tag{15}$$

Thus, $B = An - n = Af + f$. Solving these two equations gives

$$A = -\frac{f+n}{f-n}, \qquad B = -\frac{2fn}{f-n}. \tag{16}$$

1:4 • Name: Shu Haoyu
student number: 2024533012
email: shuhy2024@shanghaitech.edu.cn
Combining the X, Y, Z, and W expressions gives the matrix returned by my function:

$$P = \begin{pmatrix} q/a & 0 & 0 & 0 \\ 0 & q & 0 & 0 \\ 0 & 0 & -\dfrac{f+n}{f-n} & -\dfrac{2fn}{f-n} \\ 0 & 0 & -1 & 0 \end{pmatrix}. \tag{17}$$

The variable `s` inside `perspective` stores q; it is separate from the Phong exponent. Substituting the near and far depths into this matrix gives $z_{\text{ndc}} = -1$ and $+1$, respectively. Under the default depth mapping, the depth-buffer value is $(z_{\text{ndc}} + 1)/2$, so these planes map to 0 and 1.

2.4 support moving objects in the scene.

I use the center of the screen to select an object. A small crosshair shows the aiming position, and the picking ray starts at `eye` and follows `direction`. I use `glfwSetMouseButtonCallback` to register the mouse-button callback. Its arguments identify the window and the callback function, `MouseButtonCallback`. The callback parameters are `window`, `button`, `action`, and `mods`. They identify the window, mouse button, press or release action, and modifier-key flags. I check the left button and use the action to record its press and release.

I draw the crosshair with `glDrawArrays`. Its parameters are the primitive type, first vertex index, and vertex count; I use `GL_POINTS`, 0, and 1. The crosshair shader puts this point at the screen center, and `glPointSize(3.0f)` sets its size to 3 pixels. I temporarily call `glDisable` with `GL_DEPTH_TEST`, so the crosshair is drawn over the scene, and enable the test again afterward.

When the button is pressed, I call `OnSphere` with `eye`, `direction`, `R`, and `center`. The first two parameters define the world-space ray, and the last two specify the sphere radius and center. The function returns the nearest valid positive ray parameter `t`, or zero when there is no valid forward hit. This parameter is a distance only when the input direction is a unit vector. I use a radius of 2.55 for each sphere and 2.19 for the bunny's sphere proxy. The bunny uses this proxy rather than exact triangle intersections, so its selectable region can extend beyond its visible outline.

To obtain the sphere intersection, I write the ray as $\mathbf{x}(t) = \mathbf{e} + t\mathbf{d}$, where $\mathbf{d}$ is the nonzero input direction. For sphere center $\mathbf{c}$ and radius $R > 0$, I substitute this ray into $\|\mathbf{x} - \mathbf{c}\|^2 = R^2$. With $\mathbf{v} = \mathbf{e} - \mathbf{c}$, expansion gives

$$(\mathbf{d} \cdot \mathbf{d})t^2 + 2(\mathbf{v} \cdot \mathbf{d})t + \mathbf{v} \cdot \mathbf{v} - R^2 = 0. \tag{18}$$

Let $l = \|\mathbf{d}\|$, $m = \|\mathbf{v}\|$, and $\cos\beta = (\mathbf{v}/m) \cdot (\mathbf{d}/l)$ when $m > 0$. The same quadratic can be solved as

$$\Delta = R^2 - m^2(1 - \cos^2\beta), \qquad t_{\pm} = \frac{-m\cos\beta \pm \sqrt{\Delta}}{l}. \tag{19}$$

Here, $m \sin\beta$ is the perpendicular distance from the center to the ray's line. A negative Δ means the line misses the sphere; otherwise, I choose the smallest positive root. The division by l keeps t consistent with the original input direction. If the eye is at the center, my function handles it separately with $t = R/l$. It also uses a small tolerance around $\Delta = 0$ for near-tangent cases.

For the plane, I call `OnPlane` with `eye`, `direction`, `R`, `center`, and `normal`. Here, `R` is the square's half-size, rather than a sphere radius, and `normal` is the plane normal. I use a half-size of 1 and the normal $(0, 0, 1)$. The function returns a positive ray parameter when the intersection is in front of the camera and inside the square, and zero otherwise. Its X and Y boundary checks are intended for my current plane orientation.

For the plane equation $\mathbf{n} \cdot (\mathbf{x} - \mathbf{c}) = 0$, substituting the same ray gives

$$\mathbf{n} \cdot (\mathbf{e} + t\mathbf{d} - \mathbf{c}) = 0,$$
$$t = \frac{\mathbf{n} \cdot (\mathbf{c} - \mathbf{e})}{\mathbf{n} \cdot \mathbf{d}}. \tag{20}$$

I reject a denominator close to zero because the ray is parallel to the plane. Then I require $t > 0$ and check the hit point against $|x - c_x| \le R$ and $|y - c_y| \le R$. These square bounds apply to my plane parallel to the world XY plane.

Then I collect the returned parameters in a vector and pass it to `nearestHitIndex`. Its argument, `distances`, is this vector of candidate ray parameters. The function returns the index of the smallest positive value, or -1 if no valid hit exists. All candidates use the same ray, so I can compare these values to select the closest object.

After selecting an object, I transform its center from world space into camera space and save this position in `objectCatchPosition`. While the button remains held, I keep this saved position fixed and call `V.Inverse()`. This member function takes no parameters and returns the inverse of the current view matrix. I multiply it by the saved camera-space position to obtain the new world position. Then I update the selected object's position and model matrix. This makes the object keep the same position relative to the camera when I move or turn.

If V_0 is the view matrix when I select the object and $\mathbf{p}_{w,0}$ is its homogeneous world-space center, the saved point is $\mathbf{q} = V_0\mathbf{p}_{w,0}$. For the current frame k, I update it by

$$\mathbf{p}_{w,k} = V_k^{-1}\mathbf{q}, \qquad V_k\mathbf{p}_{w,k} = \mathbf{q}. \tag{21}$$

The second equality shows why dragging works: the center stays at the saved camera-space coordinates. Since the projection matrix stays fixed, its projected screen position and view-space depth also stay fixed.

The selected object stays selected until I release the button. On release, I clear the selection and leave the object at its new world position. This operation changes the object's position, but does not rotate its orientation with the camera.

2.5 support multiple objects in the scene.

I create separate `Mesh` instances for two spheres, a bunny, and a plane in `Main()`. The two spheres load the same mesh file, but have separate positions. I also use an `Object` structure to store each object's local center, world position, and scale.

In each frame, I set a different model matrix before drawing each object. I use `translation(pos, center)`, which returns a four-by-four matrix that moves the first point to the second point. Its displacement is `center - pos`; despite the parameter names, `pos` is the source point and `center` is the destination. For the spheres and plane, I pass the object's local center and current world position.

For the bunny, I first call `translation` to move its local center to the origin. Then I use `scale(s)`, whose single parameter is a uniform scale factor, to obtain a four-by-four scaling matrix. Finally, another translation moves the origin to the bunny's world position. I use a factor of 20. This order allows me to scale the bunny around the chosen center. Before each draw, I use `shader.Set` with the uniform name `M` and that object's model matrix.

Deriving the model matrix. For `translation(pos, center)`, let the input points be $\mathbf{a}$ and $\mathbf{b}$. The required displacement is $\boldsymbol{\delta} = \mathbf{b} - \mathbf{a}$, because $\mathbf{a} + \boldsymbol{\delta} = \mathbf{b}$. I place this displacement in the last column:

$$T(\boldsymbol{\delta}) = \begin{pmatrix} 1 & 0 & 0 & \delta_x \\ 0 & 1 & 0 & \delta_y \\ 0 & 0 & 1 & \delta_z \\ 0 & 0 & 0 & 1 \end{pmatrix}. \tag{22}$$

Then $T(\boldsymbol{\delta})(\mathbf{x}, 1)^{\mathsf{T}} = (\mathbf{x} + \boldsymbol{\delta}, 1)^{\mathsf{T}}$. Uniform scaling multiplies each spatial coordinate by the factor s, while preserving the homogeneous component:

$$S(s) = \begin{pmatrix} s & 0 & 0 & 0 \\ 0 & s & 0 & 0 \\ 0 & 0 & s & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}. \tag{23}$$

For local center $\mathbf{c}$ and desired world position $\mathbf{p}$, the bunny's model matrix is

$$M = T(\mathbf{p})S(s)T(-\mathbf{c}). \tag{24}$$

Applying the matrices from right to left gives

$$\mathbf{x}_w = s(\mathbf{x}_o - \mathbf{c}) + \mathbf{p}. \tag{25}$$

This is why I use `BunnyT2 * BunnyS * BunnyT1`. The center $\mathbf{x}_o = \mathbf{c}$ maps exactly to $\mathbf{p}$, regardless of the scale. For the spheres and plane, $s = 1$, so the model matrix reduces to $T(\mathbf{p} - \mathbf{c})$.

All these objects use the same model shader, view matrix, and projection matrix. Their separate model matrices place them at different positions. I call `glEnable` with `GL_DEPTH_TEST` to enable depth testing, so the visible surfaces are determined by their depth rather than only by the order of drawing.

2.6 support multiple lights in the fragment shader

I use one point light and one spotlight, with different colors. I define their parameters in `Main.cpp` and send them to the fragment shader through `Shader::Set`. The uniforms `pointPosition` and `pointColor` describe the point light's world position and RGB color. For the spotlight, `glightPosition`, `gColor`, and `gDirection` describe its position, RGB color, and direction from the light toward the cone. The uniform `gCosCutoff` stores the cosine of the cone's half-angle. I compute it with `std::cos`, which takes an angle in radians, so I convert 30° to radians first.

I use the same `light` function in `model.frag` for both lights. Its five parameters are `lightPosition`, `lightColor`, `worldPosition`, `N`, and `V`. They represent the light's world position and RGB color, the fragment's world position, the unit normal, and the unit direction from the fragment to the camera. This `V` is a direction vector inside the function, rather than the view matrix used by the vertex shader. The function returns a three-component color containing diffuse and specular contributions. It also uses the uniforms `material`, `ks`, and `s`.

For the spotlight, I use `normalize` to obtain unit directions and `dot` to compare the spotlight direction with the direction from the light to the fragment. If the cosine of their angle is greater than or equal to `gCosCutoff`, the fragment is inside the light cone. Otherwise, the spotlight contributes zero.

Finally, I add the point-light and spotlight contributions to the ambient term. I add ambient lighting only once, rather than once for each light. My spotlight has a half-angle of 30° and stays at a fixed world position. This version uses a hard cutoff at the cone boundary and does not include distance attenuation.

2.7 add a skybox

I implemented cubemap loading in `Texture.cpp`. The constructor `Texture(paths)` takes a vector containing exactly six image paths, in positive X, negative X, positive Y, negative Y, positive Z, and negative Z order. I create a texture ID with `glGenTextures`, whose parameters are the number of textures and the address for storing their IDs. Then I use `glBindTexture` with `GL_TEXTURE_CUBE_MAP` and the texture ID to select the cubemap.

For each face, I call `Texture::Load(path, target)`. Its parameters are the image path and the cubemap face to fill. Inside this function, `stbi_load` takes the image filename, pointers for width, height, and the original channel count, and the requested output channel count. I pass `STBI_rgb` to obtain three-channel RGB data. The function returns a pointer to the decoded pixels, or a null pointer if loading fails.

I upload the pixels with `glTexImage2D`. Its arguments specify the face target, mipmap level, internal GPU format, width, height, border, input pixel format, component type, and pixel pointer. I use level `0`, `GL_RGB8`, the loaded dimensions, border `0`, `GL_RGB`, and `GL_UNSIGNED_BYTE`. After uploading, I pass the pixel pointer to `stbi_image_free` to release the CPU image data.

I set sampling parameters with `glTexParameteri`. Its three parameters are the texture target, parameter name, and parameter value. I use `GL_LINEAR` for the minification and magnification filters, and `GL_CLAMP_TO_EDGE` for the S, T, and R wrapping modes. In `Main.cpp`, `glActiveTexture` with `GL_TEXTURE0` selects texture unit `0`. I call `Texture::Bind()`, which has no parameters and binds this texture to the cubemap target. Then I set `skyboxTexture` to integer `0` through `Shader::Set`. This value is the texture-unit index, rather than the texture object ID.

I draw a cube(which is created by myself) as the skybox. In `skybox.vert`, I use `mat3(V)` to keep the rotation part of the view matrix and convert it back to a four-by-four matrix with `mat4`. This removes the translation. Therefore, turning the camera changes the visible background direction, while translating the camera does not shift the background as if it were a nearby object. In the fragment shader, `texture` takes the cubemap sampler and `sampleDirection` and returns the sampled color. The direction comes from the cube's local position.

I draw the skybox after the scene objects. In the vertex shader, I set the clip-space Z component equal to W, which places the skybox at the farthest depth after perspective division. During this draw, I call

glDepthFunc with GL_LEQUAL. Its parameter selects the comparison rule, and this rule accepts a depth less than or equal to the stored depth. Depth testing stays enabled, so the scene objects can still cover the background. I also call glDepthMask with GL_FALSE to disable depth writes. Its boolean parameter controls writing, rather than testing. After drawing, I restore GL_LESS and enable depth writes with GL_TRUE.

3 RESULTS

The six images below show the combined lighting effects of the spotlight and point light. I show the scene from different viewing angles to compare the lighting effects. The images also show how the different light colors mix on the object surfaces and how the skybox provides the background of the scene. The blue is come from point light and the red is come from spotlight light.

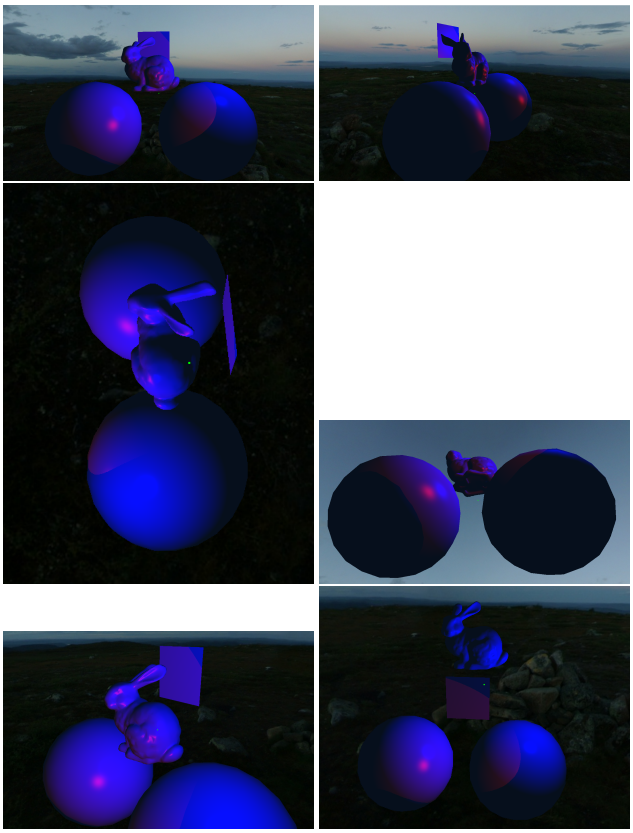


Fig. 1. phong and spotlight lightning